

Giai đoạn 1: Viết mã RTL cho Lõi Logic Analyzer

Các module RTL cần viết

1. input_synchronizer.v

- Chức năng: Đồng bộ hóa 16 kênh input vào clock domain lấy mẫu
- Chi tiết:
 - 2-stage hoặc 3-stage flip-flop synchronizer cho mỗi kênh
 - Tránh metastability khi input bất đồng bộ với sample clock
 - Ở giai đoạn nâng cao ($\geq 100\text{MHz}$): thay bằng ISERDES primitive

2. trigger_unit.v

- Chức năng: Phát hiện điều kiện trigger để bắt đầu/dừng capture
- Chi tiết giai đoạn cơ bản:
 - Chọn kênh trigger (4-bit mux select)
 - Loại trigger: Rising edge, Falling edge, Either edge, Level High/Low
 - Trigger position: Pre-trigger / Post-trigger ratio (cấu hình được)
- Nâng cao (sau):
 - Pattern trigger: so sánh toàn bộ 16 bit với mask
 - Multi-stage trigger (trigger sequencer)

3. capture_controller.v (FSM chính)

- Chức năng: State machine điều khiển quá trình capture
- Các trạng thái:
- IDLE $\rightarrow$ ARMED $\rightarrow$ PRE_TRIGGER $\rightarrow$ TRIGGERED $\rightarrow$ POST_TRIGGER $\rightarrow$ DONE
- Chi tiết:
 - IDLE: Chờ lệnh start từ PS (qua AXI register)
 - ARMED: Bắt đầu ghi dữ liệu vào buffer (circular)
 - PRE_TRIGGER: Ghi pre-trigger samples (configurable depth)
 - TRIGGERED: Trigger condition met, tiếp tục ghi post-trigger
 - POST_TRIGGER: Đếm đủ post-trigger samples
 - DONE: Báo interrupt cho PS, dừng capture

4. sample_buffer.v

- Chức năng: Bộ nhớ lưu mẫu capture

- Giai đoạn cơ bản (BRAM):
 - Dùng Block RAM, circular buffer
 - Kích thước: 64K samples $\times$ 16 bit = 128 KB (dùng ~28 BRAM 36Kb)
 - Write pointer wrap-around cho pre-trigger
 - Dual-port: Port A ghi từ capture engine, Port B đọc từ AXI
- Giai đoạn nâng cao (DDR3):
 - Thay bằng AXI4 Master $\rightarrow$ ghi vào DDR3 qua AXI Interconnect
 - Dùng AXI DMA hoặc custom AXI Master
 - Buffer depth: 1M–4M samples

5. axi_lite_registers.v

- Chức năng: Giao tiếp AXI4-Lite slave để PS cấu hình và đọc dữ liệu
- Register map đề xuất:

Offset	Tên	R/W	Mô tả
0x00	CTRL	R/W	Bit 0: Start, Bit 1: Stop, Bit 2: Reset
0x04	STATUS	R	Bit 0: Running, Bit 1: Triggered, Bit 2: Done
0x08	SAMPLE_RATE_DIV	R/W	Bộ chia tần (sample_clk = clk / (div+1))
0x0C	TRIGGER_CFG	R/W	[3:0] channel, [5:4] type, [6] enable
0x10	TRIGGER_PATTERN	R/W	16-bit pattern match
0x14	TRIGGER_MASK	R/W	16-bit mask cho pattern trigger
0x18	PRE_TRIG_COUNT	R/W	Số samples trước trigger
0x1C	POST_TRIG_COUNT	R/W	Số samples sau trigger
0x20	SAMPLE_COUNT	R	Tổng số samples đã capture

0x24	READ_ADDR	R/W	Địa chỉ đọc buffer
0x28	READ_DATA	R	Dữ liệu 16-bit tại READ_ADDR
0x2C	BUFFER_DEPTH	R	Độ sâu buffer (read-only)

6. clock_manager.v

- Chức năng: Wrapper quanh MMCM/PLL primitive
- Tạo sample clock từ system clock (200MHz onboard → chia xuống 25/50/100/200MHz)
- Cung cấp clock cho toàn bộ capture datapath

Testbench

- tb_logic_analyzer.v: Testbench tổng hợp
- Tạo stimulus 16 kênh với pattern biết trước
- Verify trigger detection, capture depth, AXI read-back
- Mô phỏng trên Vivado Simulator hoặc ModelSim

Giai đoạn 2: Tích hợp trên IP Integrator (Block Design)

Góp ý sửa đổi nhỏ

1. Giai đoạn 1 nên bắt đầu với BRAM buffer (64K samples) thay vì DDR3 → đơn giản hóa debug
2. Giai đoạn 3: Ethernet nên dùng TCP socket trên PetaLinux (lwIP hoặc Linux socket API) — đây là cách tiếp cận đơn giản nhất và ổn định nhất cho ZC702

Spec đề xuất

Thông số	Cơ bản (25MHz)	Mục tiêu (200MHz)
Kênh	16	16
Tốc độ lấy mẫu	25 MHz	200 MHz

Buffer	64K mẫu (BRAM)	1–4M mẫu (DDR3)
Trigger	Edge trên 1 kênh	Pattern + multi-stage
Truyền dữ liệu	GbE TCP/IP	GbE TCP/IP
Mức logic	3.3V LVCMOS (FMC)	3.3V qua mạch bảo vệ
Sampling method	FF synchronizer	ISERDES (DDR)

Giai đoạn 1: RTL Core

Kiến trúc gồm 6 module

input_synchronizer — 2-stage FF synchronizer cho 16 kênh, chống metastability. Khi lên 100MHz+ sẽ thay bằng ISERDES primitive.

trigger_unit — Chọn kênh trigger (mux 4-bit), hỗ trợ rising/falling/either edge và level trigger. Nâng cao: pattern match 16-bit với mask.

capture_controller — FSM chính với 5 trạng thái: IDLE → ARMED → PRE_TRIGGER → TRIGGERED → DONE. Pre-trigger dùng circular buffer, post-trigger đếm đủ samples thì dừng và báo interrupt.

sample_buffer — Dual-port BRAM circular buffer (64K × 16-bit ≈ 28 BRAM). Port A: ghi từ capture engine. Port B: đọc từ AXI. Sau này thay bằng AXI Master → DDR3.

axi_lite_registers — AXI4-Lite slave interface. Register map gồm: CTRL (start/stop/reset), STATUS (running/triggered/done), SAMPLE_RATE_DIV, TRIGGER_CFG, TRIGGER_PATTERN, TRIGGER_MASK, PRE/POST_TRIG_COUNT, READ_ADDR, READ_DATA. Khoảng 12 registers, base address configurable.

clock_manager — Wrapper MMCM primitive, nhận 200MHz system clock → tạo sample clock cấu hình được (25/50/100/200MHz).

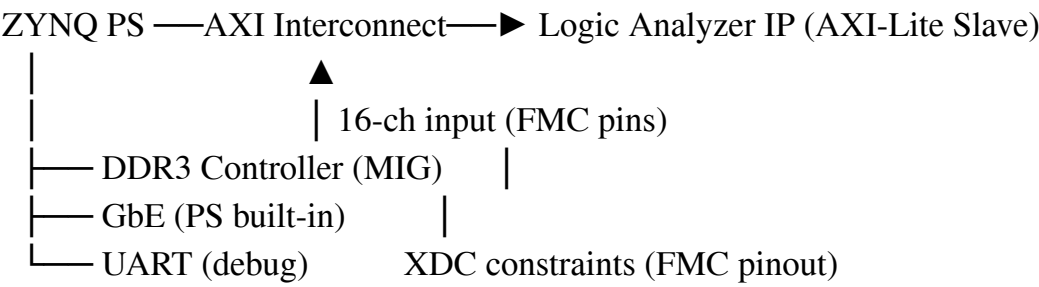
Testbench

- Tạo stimulus 16 kênh với pattern biết trước
- Verify: trigger detection chính xác, capture depth đúng, AXI readback khớp dữ liệu

- Dùng Vivado Simulator

Giai đoạn 2: Block Design trên Vivado IP Integrator

Kết nối trong Block Design

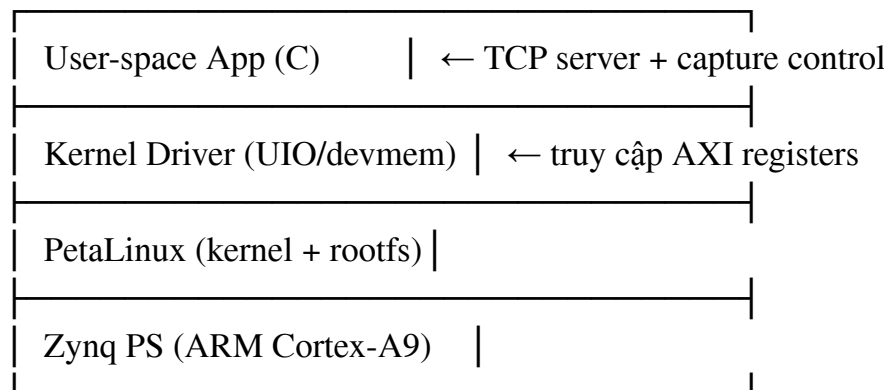


Việc cần làm

1. Package RTL thành AXI IP trong Vivado (Tools → Create and Package New IP)
2. Tạo Block Design: Zynq PS + AXI Interconnect + Logic Analyzer IP
3. Cấu hình Zynq PS: Enable GbE, DDR3, UART, M_AXI_GP0
4. XDC constraints: Map 16 kênh input vào FMC-LPC pins (tham khảo ZC702 schematic, dùng các pin FMC_LA00 → FMC_LA15)
5. Timing constraints: create_clock cho sample clock, set_input_delay cho FMC inputs
6. Generate Bitstream và export hardware (.xsa) cho PetaLinux

Giai đoạn 3: Firmware PetaLinux

Kiến trúc phần mềm



Việc cần làm

1. Tạo PetaLinux project từ .xsa export
2. Kernel driver: Dùng UIO (Userspace I/O) — đơn giản nhất cho bài tập lớn.
Thêm compatible = "generic-uio" trong device tree cho logic analyzer IP
3. Application C/C++:
 - mmap() UIO device để truy cập register
 - Hàm configure_capture(): set sample rate, trigger, depth
 - Hàm start_capture(): write CTRL register, poll/interrupt chờ DONE
 - Hàm read_buffer(): đọc toàn bộ capture buffer
4. TCP Server (cho truyền Ethernet):
 - Dùng POSIX socket API (Linux standard)
 - Protocol đề xuất: header 16 bytes (magic + command + length) + payload
 - Commands: CMD_CONFIGURE, CMD_START, CMD_STATUS, CMD_READ_DATA
 - Port: 5000 hoặc tùy chọn
 - Gửi dữ liệu capture dạng binary (16-bit per sample × N samples)

Góp ý về Ethernet

- Dùng TCP (không UDP) vì cần đảm bảo toàn vẹn dữ liệu
 - Throughput: GbE trên PetaLinux đạt ~400-600 Mbps thực tế → 64K samples × 16-bit = 128KB, truyền trong <1ms → rất thoải mái
 - Không cần bare-metal lwIP — PetaLinux TCP stack đủ hiệu năng cho use case này
-

Giai đoạn 4: Phần mềm PC (GUI)

Công nghệ đề xuất

Python + PyQt5/6 (hoặc PySide6) — phổ biến, dễ vẽ waveform, nhiều thư viện hỗ trợ.

Các tính năng cần làm

Kết nối & Điều khiển:

- Nhập IP address của board, connect/disconnect
- Cấu hình: sample rate, trigger channel, trigger type, capture depth
- Nút Start/Stop capture

Hiển thị Waveform:

- Dùng pyqtgraph hoặc matplotlib embedded — 16 kênh digital waveform dạng timing diagram
- Zoom/pan bằng chuột (scroll wheel zoom, click-drag pan)
- Cursor đo thời gian (2 cursors, hiển thị Δt và frequency)
- Grid lines với thang thời gian tự động (ns/ μ s/ms)

Phân tích:

- Protocol decoder cơ bản (tùy chọn): UART, SPI, I2C — parse từ captured data
- Export dữ liệu ra CSV hoặc VCD (Value Change Dump — tương thích GTKWave)
- Thống kê: tần số, duty cycle, pulse width

Giao diện tham khảo

Lấy cảm hứng từ PulseView (sigrok) hoặc Saleae Logic — layout gồm: toolbar trên cùng, panel kênh bên trái, waveform chính ở giữa, status bar phía dưới.

Giai đoạn 5: Analog Front-End (Mạch bảo vệ FMC)

Sơ đồ khối mạch bảo vệ

Tín hiệu vào —► [Điện trở hạn dòng]—► [Clamp diodes]—► [Buffer/Level shift]—► FMC pin
 ($\pm 30V$ max) (1k Ω –10k Ω) (BAV99/BAT54S) (SN74LVC1G17) (3.3V LVCMOS)

Thiết kế chi tiết

Tầng 1 — Điện trở hạn dòng (R_{series}):

- Giá trị: 1k Ω (cân bằng giữa bảo vệ và băng thông)
- Công suất: $\geq 0.25W$
- Tính toán: Với 30V input, $I_{max} = 30V/1k\Omega = 30mA \rightarrow$ an toàn cho clamp diodes

Tầng 2 — Clamp Diodes:

- Linh kiện: BAV99 (dual series diode) hoặc BAT54S (Schottky, nhanh hơn)
- Clamp lên VCC (3.3V) và xuống GND
- Schottky: recovery time $\sim 5ns \rightarrow$ không gây méo ở 200MHz

Tầng 3 — Buffer / Schmitt Trigger:

- Linh kiện: SN74LVC1G17 (single Schmitt trigger buffer) hoặc 74LVC245 (8-bit, cần 2 IC cho 16 kênh)
- Nguồn 3.3V, input tolerant 5V

- Propagation delay $\sim 3.5\text{ns}$ $\rightarrow$ OK cho 200MHz
- Schmitt trigger giúp clean up slow edges

Mô phỏng cần làm (LTspice hoặc TINA-TI)

1. Mô phỏng bảo vệ quá áp:
 - Input: $\pm 30\text{V}$ pulse, $\pm 50\text{V}$ (worst case)
 - Verify: điện áp sau clamp $\leq 3.6\text{V}$, dòng qua diode $<$ giới hạn
 - Kiểm tra: FMC pin voltage không vượt absolute maximum rating
2. Mô phỏng băng thông / méo dạng:
 - Input: square wave 1MHz $\rightarrow$ 100MHz $\rightarrow$ 200MHz
 - Đo rise time, fall time sau mạch bảo vệ
 - Tính bandwidth (-3dB) của toàn bộ front-end
 - Mục tiêu: BW $\geq 500\text{MHz}$ (theo Nyquist, cần $\geq 5\times$ harmonic cho square wave đẹp ở 100MHz)
 - Lưu ý: $R_{\text{series}} \times C_{\text{parasitic}}$ tạo RC filter. Với $R=1\text{k}\Omega$, $C_{\text{parasitic}} \approx 5\text{pF} \rightarrow \text{BW} \approx 32\text{MHz}$. Cần giảm R xuống 100-330 Ω nếu muốn đạt 200MHz, và thêm TVS diode (như PESD5V0S1BA) thay vì chỉ dựa vào R lớn.
3. Mô phỏng tín hiệu thực tế:
 - Đưa xung digital thực tế (có ringing, overshoot)
 - Verify dạng xung đầu ra vẫn sạch, threshold crossing đúng

Thiết kế PCB (nếu có thời gian)

- PCB 2 lớp, FMC-LPC connector (ASP-134488-01 Samtec)
- Controlled impedance 50 Ω (nếu trace dài $> 2\text{cm}$ ở 200MHz)
- Decoupling 100nF + 10 μF cho mỗi IC

Thứ tự ưu tiên & Timeline đề xuất

Tuần	Công việc	Output
1-2	GĐ1: Viết RTL + testbench, simulation pass	RTL code + sim waveform
3	GĐ2: Block Design, synthesis, bitstream	.bit + .xsa

4	GĐ3: PetaLinux + TCP server, test trên board	Boot thành công, đọc register OK
5	GĐ4: GUI PC, kết nối TCP, hiển thị waveform	App chạy, hiển thị đúng
6	GĐ5: Thiết kế + mô phỏng front-end	Schematic + sim results
7-8	Tối ưu: nâng lên 50→100→200MHz, thêm tính năng	Final demo

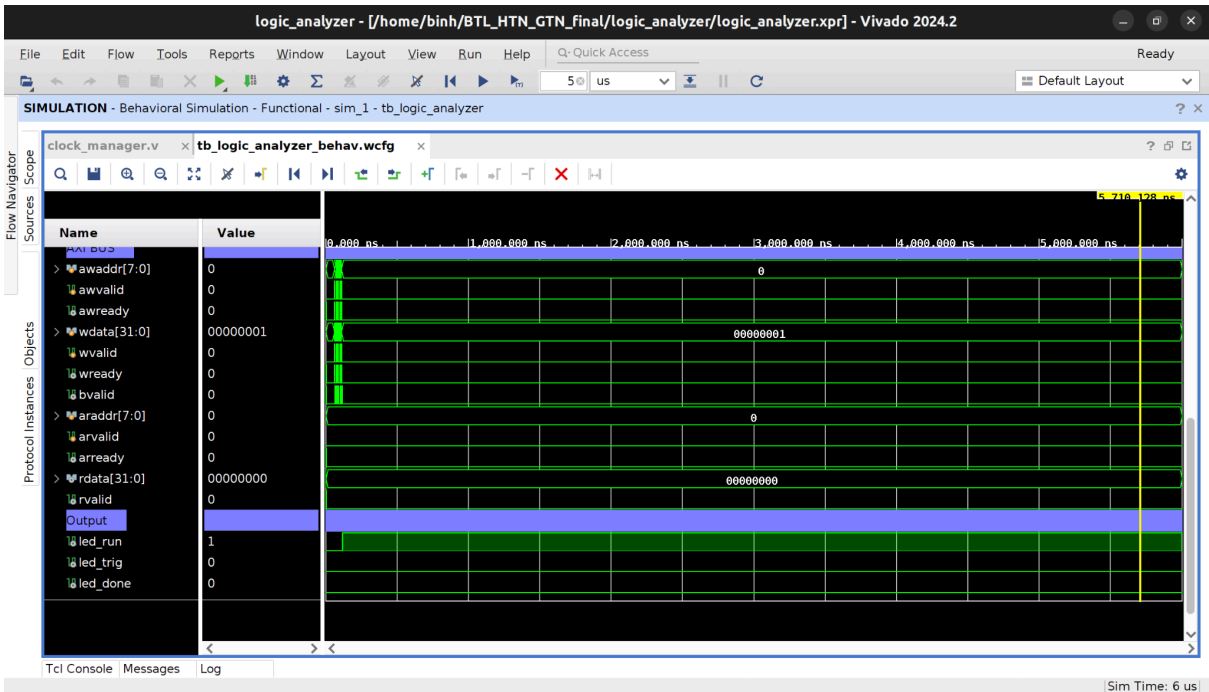
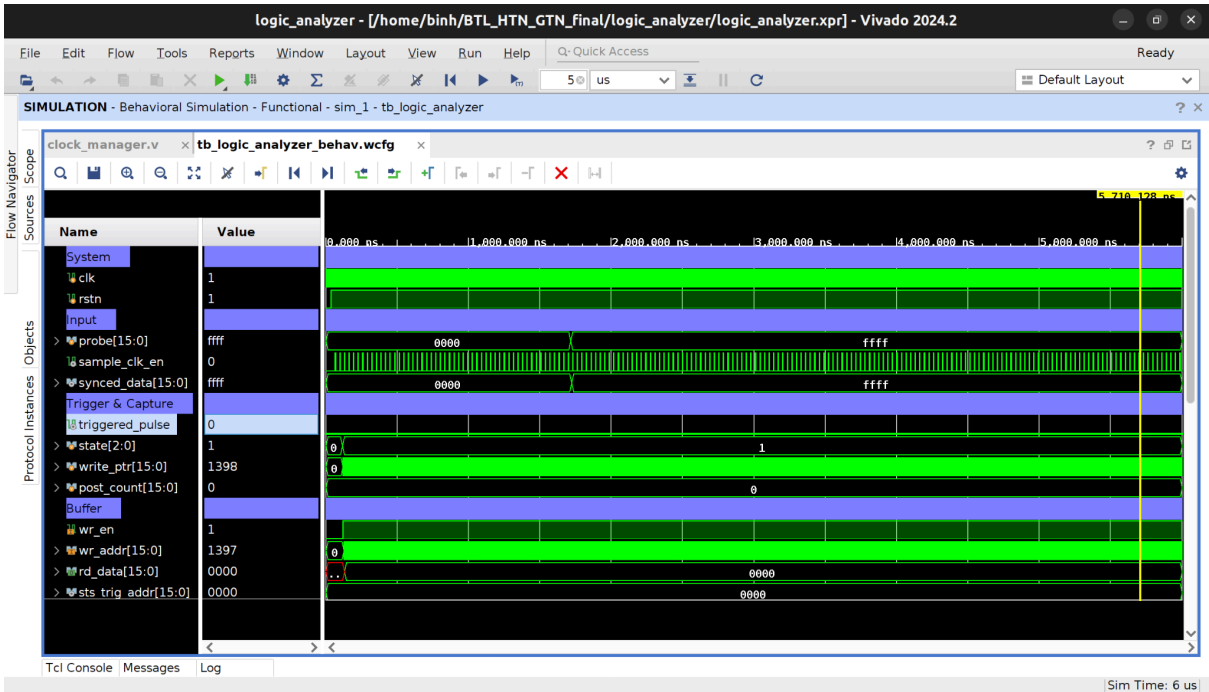
Những điểm cần lưu ý để "hoàn thành xuất sắc"

1. Tài liệu đầy đủ: register map, block diagram, timing diagram cho mỗi module
2. Simulation coverage: chứng minh RTL hoạt động đúng trước khi lên board
3. Demo thực tế: capture tín hiệu UART/SPI thật từ một MCU (Arduino chẳng hạn) → decode đúng trên GUI
4. So sánh kết quả: đặt song song capture từ project của bạn và oscilloscope/logic analyzer thương mại → chứng minh accuracy
5. Báo cáo phân tích front-end: đồ thị mô phỏng LTspice rõ ràng, chú thích đầy đủ

Khó khăn:

Giai đoạn 1:

Mô phỏng lỗi do code:



Phân tích Waveform — Phát hiện 2 Bug nghiêm trọng

Bug 1: Trigger không bao giờ kích hoạt

Quan sát: probe đã đổi từ 0x0000 → 0xFFFF (thấy rõ trên waveform $\sim 2\mu\text{s}$), nhưng triggered_pulse = 0, state vẫn ở 1 (ARMED), led_trig = 0 sau $6\mu\text{s}$.

Nguyên nhân gốc rễ — timing mismatch:

Cycle A (sample_clk_en=1): gated_data $\leftarrow$ 0xFFFF (new sample captured)

data_in = 0x0000 (output chưa update)

data_prev $\leftarrow$ 0x0000 (cập nhật = old value)

rise_edge = 0 && !0 = 0 $\leftarrow$ MISS trigger!

Cycle A+1 (sample_clk_en=0): data_in = 0xFFFF (gated_data đã update)

data_prev = 0x0000 (chưa update)

rise_edge = 1 && !0 = 1 $\leftarrow$ Đúng! Nhưng...

trig_en = 0 $\leftarrow$ sample_clk_en=0, nên KHÔNG evaluate!

Kết quả: Edge xảy ra đúng lúc trig_en = 0, nên trigger miss hoàn toàn.

Bug 2: Capture_controller ghi buffer ở 200MHz thay vì 25MHz

Quan sát: write_ptr = 1398 tại $6\mu\text{s}$ → $6\mu\text{s} \times 200\text{MHz} = 1200$ cycles → đang ghi mỗi cycle 200MHz, không phải 25MHz.

Hệ quả: 8 bản copy giống nhau của mỗi sample → lãng phí 87.5% buffer, deep capture thực tế chỉ 8K sample thay vì 64K.

Fix cụ thể cho từng file

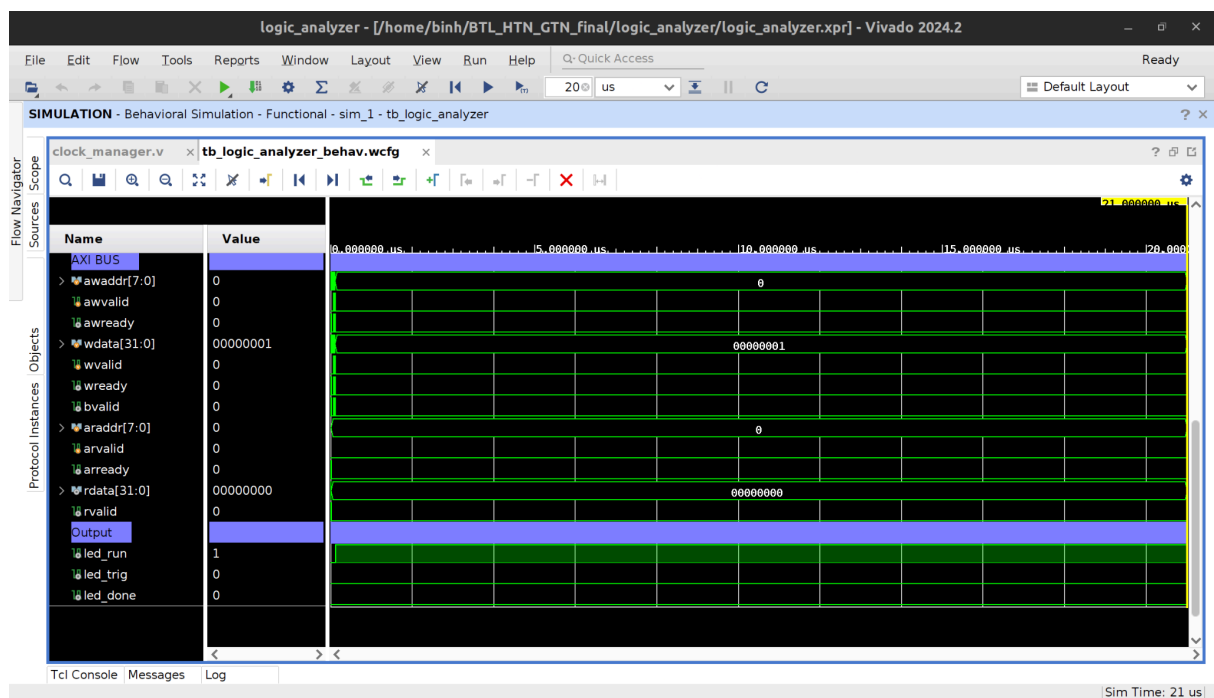
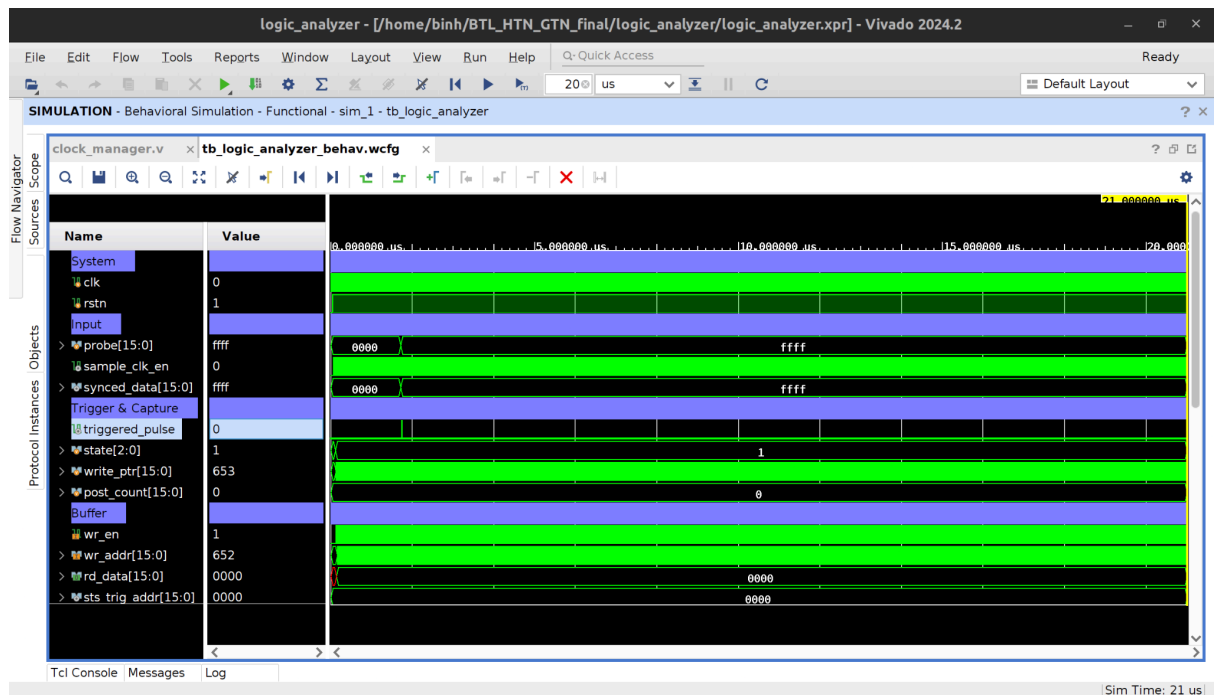
Fix Bug 1: logic_analyzer_top.v — Thêm delayed clock enable

Fix Bug 1: trigger_unit.v — Gate data_prev với trig_en

Fix Bug 2: capture_controller.v — Thêm clk_en port

Fix Bug 2: logic_analyzer_top.v — Truyền clk_en vào capture_controller

Lỗi:



Tiến triển so với lần trước

- triggered_pulse: Đã có pulse nhỏ tại $\sim 2\mu s$ (lúc probe chuyển $0x0000 \rightarrow 0xFFFF$)
→ Bug 1 đã fix đúng
- sample_clk_en: Nhịp đều hơn → Clock divider hoạt động

Bug còn lại: Điều kiện FSM không thỏa mãn

Chuỗi sự kiện thực tế:

$t \approx 0.2\mu s$: Start capture $\rightarrow$ ARMED, write_ptr bắt đầu tăng

$t \approx 2\mu s$: probe $\rightarrow 0xFFFF \rightarrow$ triggered_pulse = 1

write_ptr ≈ 50 (chỉ ~ 50 samples từ lúc start)

Điều kiện trong capture_controller:

triggered_in (=1) && write_ptr (=50) $\geq$ pre_trig_count (=256)

$\uparrow$ FALSE! $50 < 256$

$\rightarrow$ FSM KHÔNG transition $\rightarrow$ state vẫn ở ARMED

$\rightarrow$ probe cố định 0xFFFF $\rightarrow$ không còn rising edge nào nữa

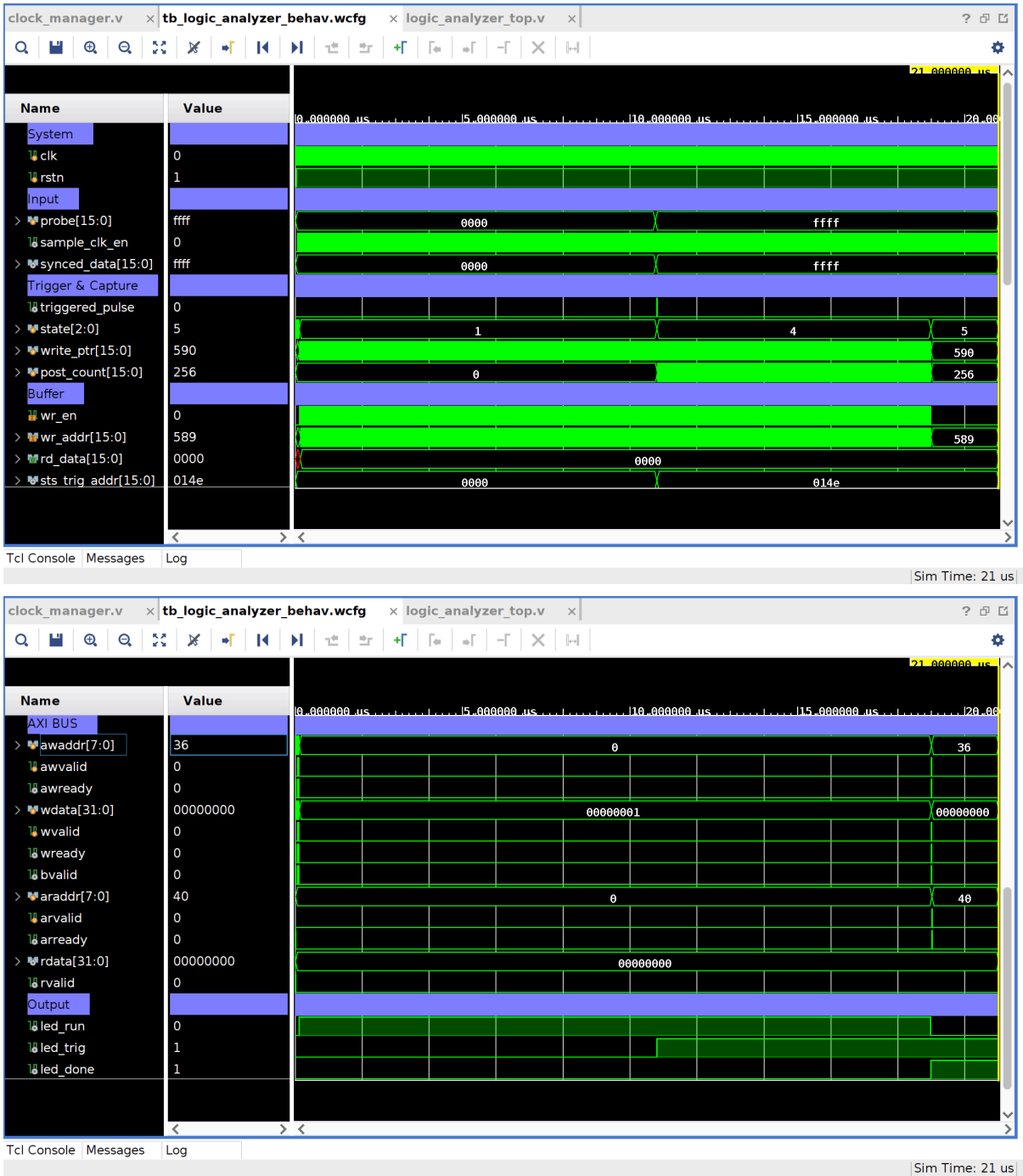
$\rightarrow$ triggered_pulse không bao giờ fire lại

$\rightarrow$ Dù write_ptr sau đó vượt 256, không có trigger $\rightarrow$ stuck mãi

Nguyên nhân: Testbench assert trigger quá sớm — chỉ có ~ 50 mẫu pre-trigger nhưng yêu cầu 256 mẫu.

Fix: Sửa testbench — Đợi đủ mẫu pre-trigger trước khi trigger

Thành công:



Signal	Giá trị	Đánh giá
probe	0x0000 → 0xFFFF tại ...	✓

triggered_pulse	Pulse nhỏ tại điểm chuyển	✓
state	1 → 4 → 5	✓ ARMED→POST_TRIG→DONE
post_count	Đếm đến 256 rồi dừng	✓
sts_trig_addr	0x014e = 334 (chốt đúng)	✓
wr_en	Về 0 sau khi DONE	✓
write_ptr	590 = 334 (pre) + 256 (post)	✓ Math đúng!
led_run	0 (kết thúc)	✓
led_trig	1	✓
led_done	1	✓
awaddr=0x24 (36)	REG_BUF_RD_ADDR	✓ Testbench đang đọc buffer
araddr=0x28 (40)	REG_BUF_RD_DATA	✓ AXI read đúng register

-> Verify thành công:

- ✓ Reset & khởi động đúng
- ✓ AXI4-Lite write/read handshake hoạt động
- ✓ sample_clk_en chia tần 200→25MHz đúng
- ✓ input_synchronizer 2-stage hoạt động
- ✓ trigger_unit phát hiện rising edge đúng
- ✓ FSM: IDLE→ARMED→POST_TRIG→DONE đúng sequence
- ✓ Pre-trigger buffer: 334 mẫu trước trigger
- ✓ Post-trigger: đếm đúng 256 mẫu

- ✓ sts_trig_addr chốt đúng địa chỉ trigger (0x014e)
- ✓ led_trig=1, led_done=1 đúng lúc
- ✓ AXI buffer readback hoạt động